



Experiment 08

Views & Materialized Views in PostgreSQL

Name: Lakshdeep UID: 24BCF10059

Aim

To study and implement PostgreSQL Views and Materialized Views, including exposing a restricted/computed column set through a view, controlling access to a view using database roles and privileges, and comparing the query performance of a live analytical query against a materialized view using **EXPLAIN ANALYZE**.

Part A – Views in PostgreSQL

A view is a stored **SELECT** query that behaves like a virtual table. A view does not store data physically; every time it is queried, PostgreSQL re-executes the underlying query against the base tables. Views are commonly used to simplify complex joins, present a controlled and computed subset of columns to different users, and enforce column-level security by hiding sensitive base-table columns.

Question 1 – Order Report View (Medium)

A company maintains customer information, product information, and sales order information in three separate tables: **customer_master**, **product_catalog**, and **sales_orders**.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount. For every order, the amounts are calculated as:

$$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$$

$$\text{Discount Amount} = \text{Gross Amount} \times \text{Discount Percentage}/100$$

$$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$$

Create a view using the three tables to generate this report. The view must expose only the five required columns and must not expose the customer's phone/email, the product's brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named **client_user** and grant this user only **SELECT** permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view's data.

Answer

Step 1 – Create the restricted, computed-column view

```
CREATE VIEW vw_order_report AS

SELECT
s.order_id,
s.order_date,
c.full_name AS customer_name,
p.product_name,

ROUND(
```

```

        (p.unit_price * s.quantity)
        - (p.unit_price * s.quantity * s.discount_percent / 100)
    , 2) AS final_amount

FROM sales_orders s

JOIN customer_master c
ON s.customer_id = c.customer_id

JOIN product_catalog p
ON s.product_id = p.product_id;

SELECT * FROM vw_order_report
ORDER BY order_id;

```

Step 2 – Create the restricted client role

```

CREATE USER client_user WITH PASSWORD 'Client@123';

-- Explicitly ensure the client has no access to the base tables
REVOKE ALL ON customer_master, product_catalog, sales_orders
FROM client_user;

-- Grant read-only access to the view only
GRANT SELECT ON vw_order_report TO client_user;

-- A join-based view is not updatable by default, but this is added
-- as an explicit safeguard so the client can never modify view data
REVOKE INSERT, UPDATE, DELETE ON vw_order_report FROM client_user;

```

Output

order_id	order_date	customer_name	product_name	final_amount
1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Part B – Materialized Views in PostgreSQL

Unlike a regular view, a materialized view physically stores the result of its defining query on disk at the time it is created. This avoids repeating expensive joins, grouping, and aggregation on every access, which is beneficial for analytical reports that do not need to reflect the base tables in real time. Because the data is a stored snapshot, a materialized view must be explicitly synchronised with the base tables using **REFRESH MATERIALIZED VIEW**; until it is refreshed, it continues to return the data that existed at the time of the last refresh.

Question 2 – Category Performance Materialized View *(Hard)*

An e-commerce company wants to regularly analyse the performance of its product categories. The database needs to perform multiple joins, **GROUP BY**, aggregate functions, **DISTINCT** counting, calculated values, and **CASE** statements to build this report. Since the same report is generated frequently, executing the full analytical query every time increases database workload.

Create a PostgreSQL materialized view named **mv_category_sales**. Also demonstrate that when new order data is inserted, the materialized view does not automatically reflect the change and must be refreshed. Finally, use **EXPLAIN ANALYZE** to compare the performance of the original analytical query with the materialized view query.

Business Rules

Requirement	Rule
Total Products Sold	SUM(quantity)
Total Orders	COUNT(DISTINCT order_id)
Unique Customers	COUNT(DISTINCT customer_id)
Total Revenue	SUM(quantity × unit_price)
Avg. Revenue per Order	Total Revenue ÷ Total Orders
Excellent	Revenue ≥ 10,000
Good	Revenue ≥ 6,000 and < 10,000
Average	Revenue ≥ 3,000 and < 6,000
Low	Revenue < 3,000

Answer

Step 1 – Category-wise sales query (with EXPLAIN ANALYZE)

```
EXPLAIN ANALYZE
SELECT
  cm.category_id,
  cm.category_name,
  SUM(od.quantity)                AS total_products_sold,
  COUNT(DISTINCT od.order_id)    AS total_orders,
  COUNT(DISTINCT od.customer_id) AS unique_customers,
  SUM(od.quantity * od.unit_price) AS total_revenue,

  ROUND(
    SUM(od.quantity * od.unit_price)::NUMERIC
    / COUNT(DISTINCT od.order_id)
    , 2) AS avg_revenue_per_order,

  CASE
    WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
    WHEN SUM(od.quantity * od.unit_price) >= 6000  THEN 'Good'
    WHEN SUM(od.quantity * od.unit_price) >= 3000  THEN 'Average'
    ELSE 'Low'
  END AS category_performance

FROM category_master cm

JOIN product_master pm
ON pm.category_id = cm.category_id

JOIN order_details od
```

```

ON od.product_id = pm.product_id

GROUP BY cm.category_id, cm.category_name

ORDER BY cm.category_id;

```

Sample Query Plan

```

GroupAggregate (cost=1.28..1.51 rows=3 width=96) (actual time=0.061..0.079 rows=3 loops=1)
  Group Key: cm.category_id
  -> Sort (cost=1.28..1.30 rows=8 width=68) (actual time=0.052..0.055 rows=8 loops=1)
    Sort Key: cm.category_id
    Sort Method: quicksort Memory: 25kB
    -> Hash Join (cost=1.11..1.16 rows=8 width=68) (actual time=0.035..0.041 rows=8 loops=1)
      Hash Cond: (od.product_id = pm.product_id)
      -> Seq Scan on order_details od (cost=0.00..1.08 rows=8 width=19)
      -> Hash (cost=1.05..1.05 rows=5 width=57)
        -> Hash Join (cost=1.02..1.05 rows=5 width=57)
          Hash Cond: (pm.category_id = cm.category_id)
          -> Seq Scan on product_master pm
          -> Hash -> Seq Scan on category_master cm

Planning Time: 0.312 ms
Execution Time: 0.146 ms

```

Step 2 – Create the materialized view

```

CREATE MATERIALIZED VIEW mv_category_sales AS

SELECT
cm.category_id,
cm.category_name,
SUM(od.quantity) AS total_products_sold,
COUNT(DISTINCT od.order_id) AS total_orders,
COUNT(DISTINCT od.customer_id) AS unique_customers,
SUM(od.quantity * od.unit_price) AS total_revenue,

ROUND(
  SUM(od.quantity * od.unit_price)::NUMERIC
  / COUNT(DISTINCT od.order_id)
, 2) AS avg_revenue_per_order,

CASE
  WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
  WHEN SUM(od.quantity * od.unit_price) >= 6000 THEN 'Good'
  WHEN SUM(od.quantity * od.unit_price) >= 3000 THEN 'Average'
  ELSE 'Low'
END AS category_performance

FROM category_master cm

JOIN product_master pm
ON pm.category_id = cm.category_id

JOIN order_details od
ON od.product_id = pm.product_id

GROUP BY cm.category_id, cm.category_name

WITH DATA;

```

Step 3 – Display data from the materialized view

```
SELECT * FROM mv_category_sales
ORDER BY category_id;
```

Output

cat_id	category	sold	orders	custs	revenue	avg/order	perf.
C1	Electronics	5	4	3	13000	3250.00	Excellent
C2	Clothing	4	3	2	10000	3333.33	Excellent
C3	Books	3	1	1	1500	1500.00	Low

Step 4 – EXPLAIN ANALYZE on the materialized view query

```
EXPLAIN ANALYZE
SELECT * FROM mv_category_sales
ORDER BY category_id;
```

```
Sort (cost=1.05..1.06 rows=3 width=96) (actual time=0.014..0.016 rows=3 loops=1)
  Sort Key: category_id
  Sort Method: quicksort  Memory: 25kB
  -> Seq Scan on mv_category_sales (cost=0.00..1.03 rows=3 width=96)
      (actual time=0.006..0.008 rows=3 loops=1)
Planning Time: 0.089 ms
Execution Time: 0.028 ms
```

Comparison

The base analytical query must perform two hash joins, a **GROUP BY** aggregation, and multiple **DISTINCT** counts every time it runs. The materialized-view query, in contrast, is a single sequential scan over an already-computed result set, so its plan is far simpler and its execution time is consistently lower. On this small sample dataset the absolute difference is a fraction of a millisecond, but on a production-sized **order_details** table the saving becomes substantial, which is the main motivation for using a materialized view for a frequently-run report.

Step 5 – Insert new order data and observe stale results

```
INSERT INTO order_details VALUES
('0109','U1','P202',2,2000);

-- Querying the materialized view immediately still returns
-- the OLD (stale) data, since it has not been refreshed yet
SELECT * FROM mv_category_sales
WHERE category_id = 'C2';
```

Output (before refresh – stale)

cat_id	category	sold	orders	custs	revenue	avg/order	perf.
C2	Clothing	4	3	2	10000	3333.33	Excellent

Step 6 – Refresh the materialized view

```
REFRESH MATERIALIZED VIEW mv_category_sales;
```

```
SELECT * FROM mv_category_sales  
WHERE category_id = 'C2';
```

Output (after refresh – updated)

cat_id	category	sold	orders	custs	revenue	avg/order	perf.
C2	Clothing	6	4	2	14000	3500.00	Excellent

The **REFRESH MATERIALIZED VIEW** statement re-executed the defining query and replaced the stored snapshot, so the new order (O109) is now correctly reflected: total products sold increased from 4 to 6, total orders from 3 to 4, and total revenue from 10,000 to 14,000.

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how a **VIEW** exposes a restricted, computed set of columns while hiding sensitive base-table data.
- Learned to manage database access control by creating a PostgreSQL role and granting it column/row-restricted, read-only privileges through **GRANT** and **REVOKE**.
- Implemented multi-table joins with derived/calculated columns (gross amount, discount amount, final amount) inside a view definition.
- Understood the difference between a regular view (always live) and a **MATERIALIZED VIEW** (a stored, physical snapshot of query results).
- Learned to build an analytical report using multiple joins, **GROUP BY**, **DISTINCT** counts, and business-rule classification with **CASE**.
- Used **EXPLAIN ANALYZE** to compare the query execution plan and performance of a live analytical query against a materialized view.
- Observed that a materialized view becomes stale after base-table changes and must be explicitly synchronised using **REFRESH MATERIALIZED VIEW**.